

EXPERIMENT 1

Low-Code vs Traditional Application Development

A comparative study of two ways to turn an idea into working software

Student	Maitreyee DD	Roll No.	241401050
Department	Computer Science and Business Systems	Experiment	01
Subject	Low-Code Application Development using Zoho Creator	Contact hours	2

1 Aim

To study and compare **low-code development** with **traditional (hand-coded) application development** in terms of process, speed, cost, flexibility, security and scalability, and to understand when each approach should be chosen.

Learning outcomes. After this experiment I should be able to (a) describe the lifecycle of both approaches, (b) justify a choice between them for a given business problem, and (c) explain where Zoho Creator sits in the low-code landscape.

2 Background

Every organisation eventually needs software: a form to collect requests, a dashboard to see numbers, a workflow to route approvals. Historically, the only way to get such a tool was to hire programmers who wrote it line by line. As the demand for business apps grew faster than the supply of developers, vendors began shipping **visual platforms** where an application is assembled from configurable building blocks. This created the two broad paths compared in this report.

Key terms. *Hand-coding* means writing source code in languages such as Java, Python or C#. *Low-code* means building most of the application through visual modelling and configuration, writing code only for special rules. *Citizen developer* means a business user without a formal programming background who builds apps on such a platform.

3 Traditional Application Development

In the traditional approach, a team of developers designs the architecture and writes the user interface, business rules, database access and server code themselves. Frameworks and libraries help, but the team still assembles and owns every layer. The work typically follows a lifecycle (SDLC), shown below.

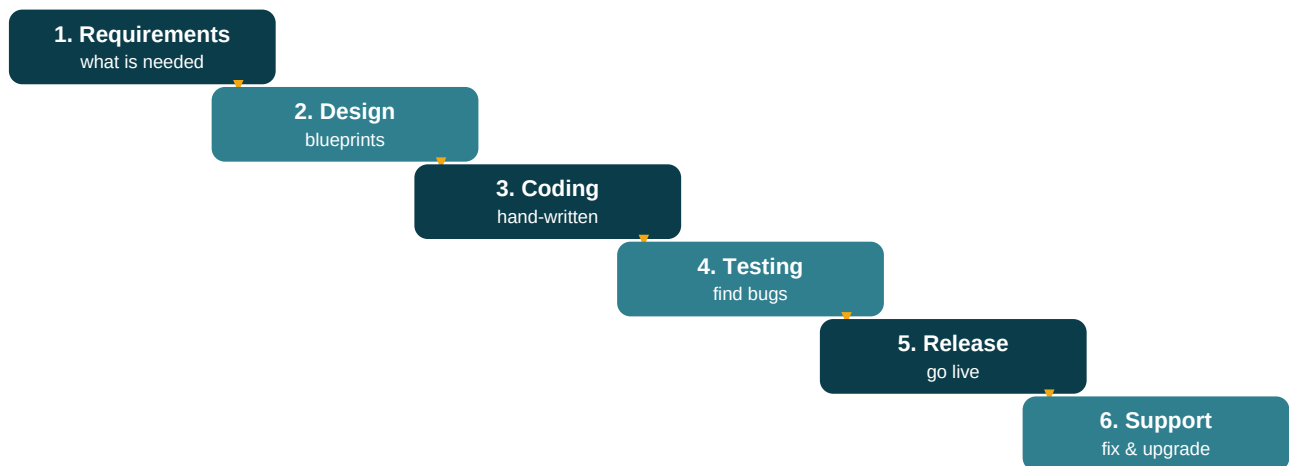


Figure 1: Waterfall-style lifecycle of a hand-coded project; each stage hands over to the next.

- **Strength:** total freedom - any feature, integration or performance tuning is possible.
- **Cost:** needs skilled developers, testers, DevOps staff and a long timeline.
- **Ownership:** the organisation owns the code and controls where and how it runs.

4 Low-Code Application Development

A low-code platform bundles the common layers of an application (database, hosting, authentication, UI toolkit) and exposes them through a visual designer. The developer describes *what* the app should do by designing forms, arranging reports and defining workflows, while the platform generates and runs the underlying code.

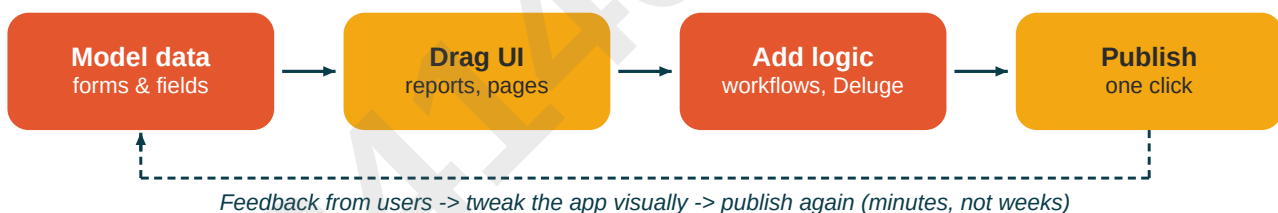


Figure 2: The low-code build loop. Changes are made visually and republished quickly.

Example - Zoho Creator. In Zoho Creator, a developer creates *forms* to model data, builds *reports* and *dashboards* through the UI builder, automates processes with *workflows* and the *Deluge* scripting language, and publishes the app to web and mobile from the same project. Small pieces of Deluge cover the few custom rules that drag-and-drop cannot, which is exactly what "low" rather than "no" code means.

Traditional: you build every layer

User interface	built by you
Business logic	built by you
Database	built by you
Server & hosting	built by you
Security & login	built by you

Low-code: platform supplies most layers

User interface	configured by you
Business logic	configured by you
Database	provided by platform
Server & hosting	provided by platform
Security & login	provided by platform

Figure 3: Who is responsible for which layer. Low-code shifts infrastructure work to the vendor.

5 Where the Effort Goes

The chart below sketches how effort is typically distributed across the main activities of a small business application. The numbers are illustrative estimates meant to show the pattern, not measured data.

Relative effort (person-days, illustrative for a small business app)

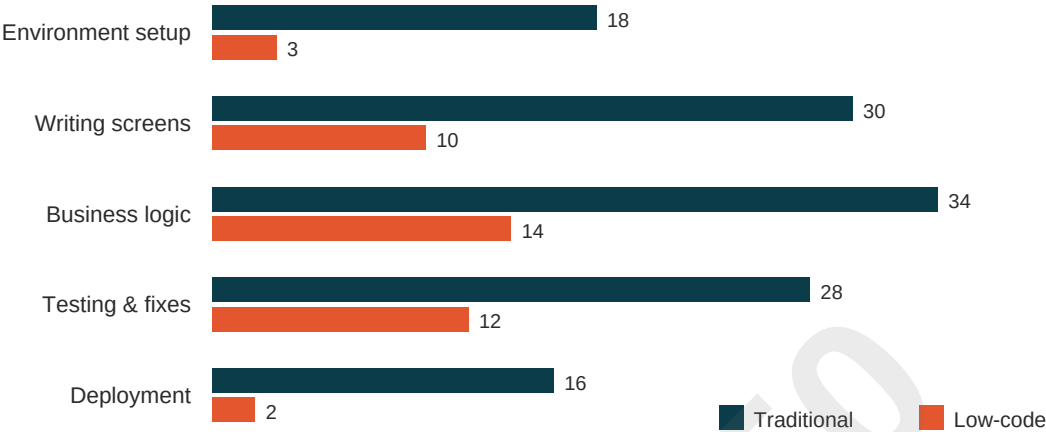


Figure 4: Illustrative effort comparison. Low-code saves the most in setup, deployment and screen building.

6 Detailed Comparison

Factor	Traditional development	Low-code development
Speed of delivery	Weeks to months	Days to a few weeks
Code written	Almost everything	Mainly custom rules and integrations
Skills needed	Professional programmers	Analysts and citizen developers, plus a few coders
Customisation	Unlimited	Bounded by platform features and extension points
Upfront cost	High (large team, tooling, servers)	Low to medium (subscription per user or app)
Long-term cost	Maintenance staff, infrastructure	Licence fees that grow with users
Performance tuning	Full control over code and database	Limited to platform settings
Security	Designed and audited by the team	Shared responsibility; vendor secures the platform
Scalability	Can be engineered for very high loads	Good for most business loads; platform limits apply
Maintenance	Developers patch and upgrade	Vendor handles platform upgrades
Deployment	Servers, pipelines and manual release	Built-in publish, environments and versioning
Vendor dependence	Low (open tools, portable code)	High (apps tied to the platform)
Change requests	Requires code change, retest, redeploy	Edit visually and republish

7 Advantages and Disadvantages

Low-code development

Advantages

- Quick prototypes and quick releases
- Fewer people and lower initial cost
- Business users can take part in building
- Built-in hosting, security and versioning
- Easy to change when requirements shift

Disadvantages

- Locked into one vendor's ecosystem
- Hard to build highly unusual features
- Subscription costs rise with usage
- Less control over performance internals
- Platform limits on data volume and API calls

Traditional development

Advantages

- Complete freedom over design and tech stack
- Best fit for performance-critical systems
- Scales to millions of users when engineered well
- Code and data are fully portable
- Security can be tailored to strict regulations

Disadvantages

- Long time before the first usable version
- Expensive team and infrastructure
- Steep skill requirement
- Ongoing maintenance burden
- Slow to react to changing requirements

8 Everyday Analogies

Tailor versus ready-to-wear. A tailor measures you and cuts cloth from scratch, giving a perfect fit but taking days and costing more. Ready-to-wear with minor alterations is available the same day and fits most people well. Traditional development is the tailor; low-code is the alteration shop.

Own a car versus use a cab service. Owning a car lets you modify the engine and drive anywhere, but you handle fuel, servicing and insurance. A cab gets you there quickly with none of that worry, though you follow the operator's routes and pricing. The cab is the low-code platform; the vendor maintains the vehicle.

9 Case Study: Hostel Maintenance Request Tracker

A college hostel wants a system where residents raise maintenance requests (water, electrical, furniture), wardens assign them to staff, and students are notified when a request is closed. The table compares how the project might unfold under each approach (illustrative planning estimates).

Aspect	Traditional	Low-code (Zoho Creator)
Time to first working version	8 to 10 weeks	About 1 week
Team	3 to 5 (developer, tester, designer)	1 to 2 (one builder plus warden for feedback)
Data model	Schema design, ORM, migrations	Forms for Request, Staff, Block with lookup fields
Assignment and alerts	Custom backend and email service	Workflow plus email/notification actions
Dashboards	Separate charting library	Built-in reports and dashboard components
Hosting and release	Set up server and pipeline	Publish from the platform

Observation. For a department-level tool with standard forms, approvals and reports, low-code reaches a usable product several times faster. If the college later wants machine-learning based prediction of faults or integration with a custom campus IoT network, a hybrid path (Zoho Creator plus custom functions) or a coded service would be justified.

10 Choosing the Right Approach

The simple decision flow below summarises the reasoning used in this study.

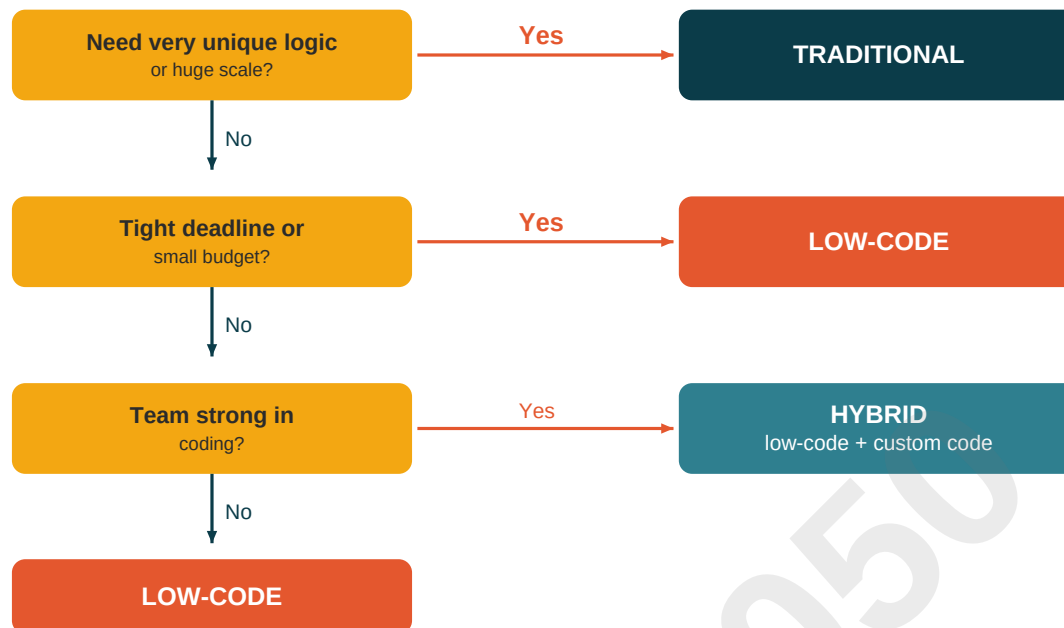


Figure 5: A quick decision guide. Real projects may mix both approaches.

The hybrid reality. Modern platforms are not closed boxes. Zoho Creator supports custom functions and REST APIs, and can be extended with pro-code services such as Zoho Catalyst, so teams can keep speed for 80 percent of the app and write real code only where it truly pays off.

11 Conclusion

Low-code and traditional development are not rivals so much as tools with different sweet spots. Low-code wins on speed, cost and accessibility, and is the sensible default for internal tools, workflows and prototypes. Traditional development wins when uniqueness, extreme scale, deep performance tuning or strict control are non-negotiable. A good engineer evaluates the problem first and then picks the approach, or combines both.

Result. The comparative study of low-code and traditional application development was completed, and a decision guide for selecting the appropriate approach was prepared.

References

- Zoho Creator documentation - help.zoho.com/portal/en/kb/creator
- ACM Digital Library - "Low-Code Development Platforms: A Systematic Review"
- Gartner Research - Magic Quadrant for Enterprise Low-Code Application Platforms
- IEEE Transactions on Software Engineering - articles on rapid application development and low-code